Project Proposal: AXI-Based Streaming FFT Accelerator with Internal DMA

Note: We are not using/making an actual CPU we will testbench via writing to registers to mimic what a CPU would do

1. Project Subject

This project is centered around an FFT Accelerator that uses AXI-4 streaming to a DMA. The DMA moves data from main memory and the accelerator with minimal intervention of our hypothetical CPU, allowing the "CPU" to configure parameters at runtime via registers while the accelerator continues to run. The accelerator would implement the Fast Fourier Transform to reduce CPU workload allowing the CPU to simply write to the configuration registers to enable the use of the FFT Accelerator. To continue to move data without CPU involvement, we will utilize a DMA and AXI-4 Stream to continue streaming data to the FFT for processing and calculations.

The goal here is to create a system that is a dedicated hardware module that allows the CPU to offload FFT computations to it and run independently and optimize for high throughput.

2. High-Level Operation

We will create a simulated environment and a DMA engine. This DMA will handle transfers of data from memory to the accelerator and to the CPU and data registers. It will support double buffering while streaming in data to the FFT core to ensure that streaming and processing are continuous with minimal gaps with minimized stalling so that we can maximize throughput.

The FFT Accelerator Core processes input frames of configurable size (512 default/max), and applies pipelined butterfly operations. It will then stream results back via an AXI stream back to the DMA. We will design for a parameterizable size and use fixed-point math.

The system will be able to set FFT parameters during runtime and new frames/configurations will only be applied at the start of a new FFT frame to prevent corruption using either a frame counter or a last signal that comes with the data. This allows for flexible operation. Our system will also have additional signals such as status and error signals to help with things such as backpressure and signify overflow respectively.

Our overall architecture contains a DMA, AXI-4 Streaming Manager, and the FFT Accelerator. Our DMA internal registers will be read/write from the testbench since we don't have an actual CPU.

Our system will function as the following: The "CPU" will configure the DMA to use the streaming manager to send over values to the FFT configuration registers and enable the FFT. The CPU will then configure the DMA's internal registers to access the correct memory addresses of the external-memory where the FFT data itself lies and do the proper transfer. The CPU then triggers the DMA to stream the data from memory into the AXI-4 Streaming Manager's memory. Lastly, the streaming manager can stream to the FFT Accelerator's input register where the core will implement the FFT computation. Then the numbers can be streamed back to the DMA who will write to memory. Our system will be runtime configurable so that parameters can be changed without interrupting workflow and causing backlog.

4. The FFT Core

Fast Fourier Transform (FFT) is a math algorithm that computes the Discrete Fourier Transform of a signal efficiently. This is similar to the FFT button on oscilloscopes.

The DFT of an x[n] is defined as

$$X[k] = \sum_{n=0}^{N-1} x[n] * e^{\frac{-2j\pi kn}{N}}, \quad k = 0, 1, \ldots, N-1$$

We will likely use the common Cooley-Tukey DIF radix-2 FFT approach meaning we will implement a radix-2 size where we combine results using butterfly operations. DIF splits the output frequency into even and odd indexed groups instead of the input. Our butterfly operation is then as follows

$$y_0 = a + b$$
$$y_1 = (a - b) * W^k_N$$

Here the W term is our twiddle factor that represents $e^{\frac{-2j\pi k}{N}}$. y_0 and y_1 are the transformed values. Twiddle factors are precomputed and stored as 16-bit Q1.15 fixed point values.

Our FFT module will have a default frame size of 512 meaning that it will process 512 inputs at a time and as a result will have 9 stages. Each stage will perform a butterfly operation. Additionally each stage contains delays to wait for the correct pairings most likely using shift registers. The last stage pairs adjacent numbers together, the second to last stage waits for numbers that are one more apart. For example if we have x_0, x_1, x_2, x_3 the second stage would pair together x_0 and x_2 as its input operands. Stage delays can be calculated with delay = $\frac{N}{2^{stage}}$ Each stage processes one new input per clock cycle. Internally, they will keep track using delay lines of how it grabs and does the math on the correct number.

The complex addition and multiplication can all be handled by 4 real multipliers and 6 real adders/subtractors per unit but we will simply use two different registers to store complex numbers one for the real and one for the imaginary or figure out some other form of storage as we optimize our system. Twiddle factors themselves can be precomputed and stored and then accessed with a simple lookup table.

We plan to use FSM Control logic to control our FFT system like waiting for the CPU to start it, capturing config registers, running the system, and streaming back.

DIF produces outputs in bit-reversed order. For our project, we are going to choose to simply write this bit-reversed order value back into the output. This is to keep our project lightweight and reduce the need for expensive memory sitting on our accelerator. In a real system, this reversal could all be done in software which is a lot cheaper than having this additional memory although doing this in hardware is an option if the project complexity is not high enough.

Registers:

Start Register
Address for DMA to read from
Address for DMA to write to
Status
Overflow_Flag
Frame Size
Number of Frames / Continuous Mode

3. Division of Labor (Possibility for modification depending on how things go)

	Tasks
Person A	FFT Core
Person B	DMA + Environment + Streaming Input
Person C	FFT Control + Streaming Output

4. Stretch Goals

1. FFT Size Switching
    a. Support multiple FFT radix sizes runtime to reduce latency for larger FFTs
2. Windowing Support(Hann, Hamming)
    a. add support for common FFT windows. Coefficients will be stored in memory with an additional register for selecting a type.
    b. hardware multiplies each incoming sample by the necessary window coefficient before it enters the FFT stages

3. Real Time Magnitude/Phase Output
    a. Add hardware that computes magnitude and phase. Use a CORDIC approximation to compute square roots.
4. Runtime Configuration

5. Fundamental Requirements
    1. AXI Stream Interface
        a. Correctly implement the AXI-4 Stream Protocol on input and output interfaces. The system must correctly handle TVALID/TREADY handshake for backpressure, use TLAST to mark the final sample of the frame, and do not corrupt the data during the transfer.
    2. FFT Computation
        a. FFT core must correctly produce a mathematically correct frequency-domain output for a given time-domain input. We will verify this by feeding in a dataset and comparing the output that we get with a FFT calculation from a verified source such as feeding a pure sine wave.
    3. DMA Data Transfer
        a. DMA must read a full 512-sample frame from simulated memory and stream it to the FFT core via the AXI Stream after being triggered by the "CPU". Upon the FFT's completion it must write the 512 output samples back to the correct address in memory.
    4. Runtime Register Configuration
        a. The system must allow the "CPU" to write to configuration registers at any time without corrupting the FFT's frame. New variables must only take effect after the current frame has finished processing.
    5. Overflow Detection DEALT IT
        a. The system must detect fixed-point overflow and set an overflow status when this occurs. This flag must be readable by the CPU.
    6. Integration
        a. The entire system must be integrated together and function as stated above as a whole with the full high-level flow working under different configurations.

We are EXTREMELY hoping for advice/suggestions about project changes, suggestions, and improvements to make this a proper project or make it more doable in the time constraint.